Flight & Airline Database System (SQL + Python)

Module Title and Code: Introduction to Databases, UFCFTK-30-1

Submitted By: Raunak Kumar Singh

Student Number: 24071246

Orbund Id: 11281

Submitted To: Rabin Gautam

TASK 2 (30%)

1. Consider the following **FLIGHT** table given below:

id	source	destination	aid	fare	discount	capacity
RA230	Kathmandu	Delhi	1	50000		200
AI215	Delhi	Kathmandu	2	60000	5000	300
3L046	Kathmandu	Abu Dhabi	3	70000		300
RA416	Kathmandu	Kuala Lumpur	1	55000	2000	180
H9891	Kuala Lumpur	Kathmandu	4	65000	3000	250

Similarly consider the following **AIRLINES** table given below:

id	name	country	founded_year	fleet_size
1	Air Nepal	Nepal	1990	20
2	Air India	India	1932	150
3	Etihad Airways	UAE	2003	120
4	Malaysia Airlines	Malaysia	1947	80

1. Write SQL syntax to create database with name **TRACKER** and the given tables: **FLIGHT** and **AIRLINES**. The **aid** column must refer to **id** of **AIRLINES** table.

Answer:

```
MariaDB [(none)]> CREATE DATABASE TRACKER;
Query OK, 1 row affected (0.002 sec)

MariaDB [(none)]> USE TRACKER;
Database changed
MariaDB [TRACKER]> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| company_db         |
| employees          |
| information_schema |
| mysql              |
| nepal              |
| performance_schema |
| phpmyadmin         |
| test               |
| tracker            |
+--------------------+
9 rows in set (0.001 sec)

MariaDB [TRACKER]>
```

Firstly, the database **TRACKER** was created.

```
MariaDB [TRACKER]> CREATE TABLE AIRLINES (
    ->     id INT PRIMARY KEY,
    ->     name VARCHAR(50) NOT NULL,
    ->     country VARCHAR(50) NOT NULL,
    ->     founded_year INT NOT NULL,
    ->     fleet_size INT NOT NULL
    -> );
Query OK, 0 rows affected (0.020 sec)

MariaDB [TRACKER]> CREATE TABLE FLIGHT (
    ->     id VARCHAR(10) PRIMARY KEY,
    ->     source VARCHAR(50) NOT NULL,
    ->     destination VARCHAR(50) NOT NULL,
    ->     aid INT NOT NULL,
    ->     fare DECIMAL(10, 2) NOT NULL,
    ->     discount DECIMAL(10, 2) DEFAULT NULL,
    ->     capacity INT NOT NULL,
    ->     FOREIGN KEY (aid) REFERENCES AIRLINES(id)
    -> );
Query OK, 0 rows affected (0.042 sec)
```

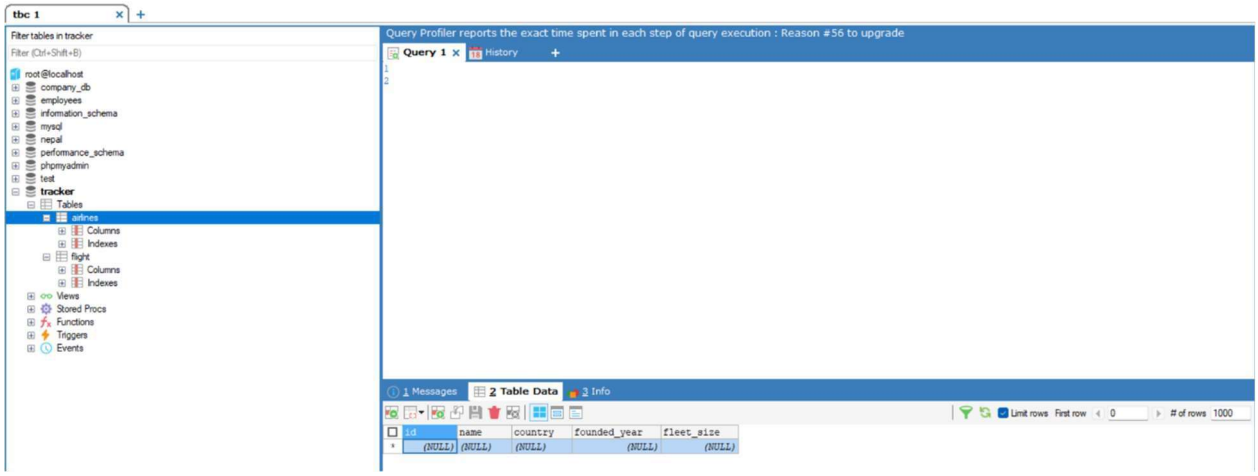
After that, the **AIRLINES** table was created first, then I created **Flight** table which ensures foreign key referencing (**aid – AIRLINES.id**).

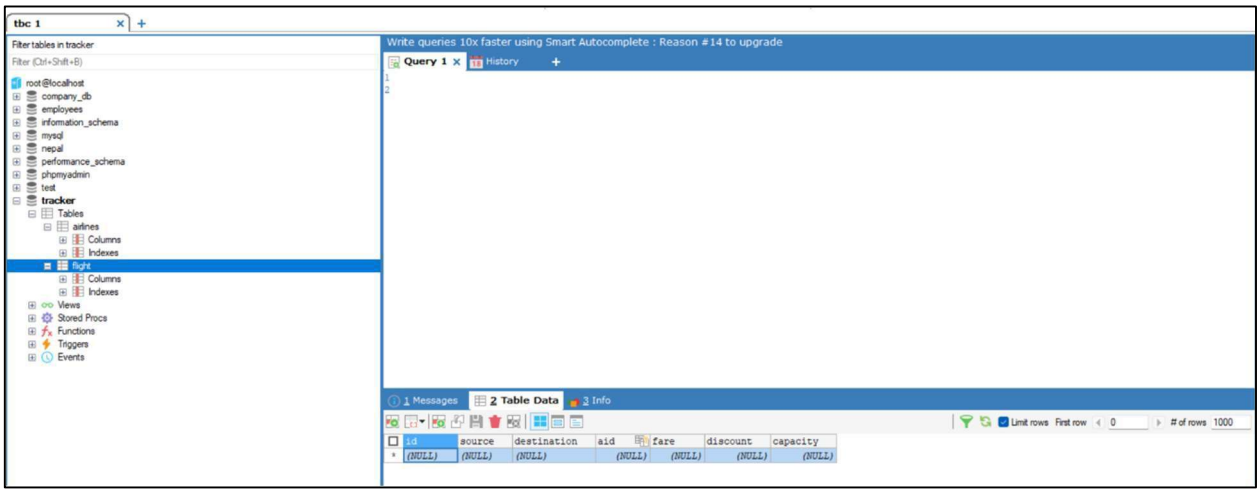
At the beginning, I started the **XAMPP Server** to run MYSQL in my own local system. After that, I opened **SQLyog** and connected it to the localhost database server.

Then, I went back to XAMPP Server and used its terminal to access MYSQL by running the command "**mysql -u root -p**".

After the connection was achieved, I created a database named **TRACKER** as per the question.

Now at first, I created the table for **AIRLINES**. Then, I created the table for **FLIGHT** and added a foreign key in it so that the **aid** column will refer to the **id** column in the AIRLINES table.





At first both tables were empty.

2. Connect this database: **TRACKER** with Python and create a Python script that inserts at least 5 rows with random data into the FLIGHT and AIRLINES table.

Answer:

The **TRACKER** database was connected using **Python in Jupyter Notebook** (via VS Code).

```python
import mysql.connector
print("Working")
```

[1] ✓ 0.1s

```
Working
```

```python
def connect_database():
    try:
        conn = mysql.connector.connect(
            host="localhost",
            user="root",
            password="Raunak@9801",
            database="TRACKER"
        )
        print("Successfully connected to database!")
        return conn
    except mysql.connector.Error as err:
        print(f"Error: {err}")
        return None
```

[2] ✓ 0.0s

```python
def insert_sample_data(connection):
    try:
        cursor = connection.cursor()


        airlines_data = [
            (1, 'Air Nepal', 'Nepal', 1990, 20),
            (2, 'Air India', 'India', 1932, 150),
            (3, 'Etihad Airway', 'UAE', 2003, 120),
            (4, 'Malaysia Airlines', 'Malaysia', 1947, 80),
            (5, 'Qatar Airways', 'Qatar', 1993, 200)
        ]

        cursor.executemany(
            "INSERT INTO AIRLINES VALUES (%s, %s, %s, %s, %s)",
            airlines_data
        )


        flight_data = [
            ('RA230', 'Kathmandu', 'Delhi', 1, 50000, None, 200),
            ('AI215', 'Delhi', 'Kathmandu', 2, 60000, 5000, 300),
            ('3L046', 'Kathmandu', 'Abu Dhabi', 3, 70000, None, 300),
            ('RA416', 'Kathmandu', 'Kuala Lumpur', 1, 55000, 2000, 180),
            ('H9891', 'Kuala Lumpur', 'Kathmandu', 4, 65000, 3000, 250)
        ]

        cursor.executemany(
            """INSERT INTO FLIGHT
            (id, source, destination, aid, fare, discount, capacity)
            VALUES (%s, %s, %s, %s, %s, %s, %s)""",
            flight_data
        )
```

```python
        connection.commit()
        print("Successfully inserted 5 rows in each table!")

    except mysql.connector.Error as err:
        print(f"Error inserting data: {err}")
        connection.rollback()
    finally:
        if cursor:
            cursor.close()
```

✓ 0.0s

```python
if __name__ == "__main__":
    db_connection = connect_database()
    if db_connection:
        insert_sample_data(db_connection)
        db_connection.close()
        print("Database connection closed")
```

✓ 0.0s

```
Successfully connected to database!
Successfully inserted 5 rows in each table!
Database connection closed
```

After implementing the script, the data was successfully added.

To verify the implementation of the queries, SQLyog was used to view the tables.

1 Messages | 2 Table Data | 3 Info

id	name	country	founded_year	fleet_size
1	Air Nepal	Nepal	1990	20
2	Air India	India	1932	150
3	Etihad Airway	UAE	2003	120
4	Malaysia Airlines	Malaysia	1947	80
5	Qatar Airways	Qatar	1993	200
(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

1 Messages | 2 Table Data | 3 Info

id	source	destination	aid	fare	discount	capacity
3L046	Kathmandu	Abu Dhabi	3	70000.00	(NULL)	300
AI215	Delhi	Kathmandu	2	60000.00	5000.00	300
H9891	Kuala Lumpur	Kathmandu	4	65000.00	3000.00	250
RA230	Kathmandu	Delhi	1	50000.00	(NULL)	200
RA416	Kathmandu	Kuala Lumpur	1	55000.00	2000.00	180
(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

For this task, I connected the **TRACKER** database with **Python using Jupyter Notebook within VS Code**.

At first, I installed and imported the required **Python Library (PyMySQL)** to make the connection between python and MySQL. Then, I wrote the connection code using the **host, username, password, and database name**.

Next, I created a python code to insert at least five records into both the **AIRLINES** and **FLIGHT** tables. I used **SQL INSERT** inside the python code and implemented them through the connection built within the system.

After inserting data, I committed the changes to make sure the records were saved properly in the system.

For verification, I opened **SQLyog** and refreshed my current data base and after that all the data were inserted into the tables in the TRACKER database.

For Q-3, I used **XAMPP** Server and its terminal to access **MySQL** and **SQLyog** to run the following queries. I implemented different SQL commands according to the respective questions to **retrieve**, **update**, and **filter data**.

3. Execute the following using SQLyog or any alternatives DBMS:

a. SQL query to list all the flights of source to destination on which flight is capable to hold maximum passengers.

Answer:

```
SELECT source, destination, MAX (capacity) AS maximum
FROM FLIGHT
GROUP BY source,
destination ORDER BY
maximum DESC;
```

```
MariaDB [TRACKER]> SELECT source, destination, MAX(capacity) AS maximum
    -> FROM FLIGHT
    -> GROUP BY source, destination
    -> ORDER BY maximum DESC;
+--------------+--------------+---------+
| source       | destination  | maximum |
+--------------+--------------+---------+
| Delhi        | Kathmandu    |     300 |
| Kathmandu    | Abu Dhabi    |     300 |
| Kuala Lumpur | Kathmandu    |     250 |
| Kathmandu    | Delhi        |     200 |
| Kathmandu    | Kuala Lumpur |     180 |
+--------------+--------------+---------+
5 rows in set (0.001 sec)
```

This query obtains the data of the flights between each source and destination with the maximum passenger capacity.

b. SQL query to retrieve the total flights that don't take off from Kathmandu.

Answer:

SELECT COUNT (*) AS noFlightKTM

FROM FLIGHT WHERE source! = 'Kathmandu';

```
MariaDB [TRACKER]> SELECT COUNT(*) AS noFlightKTM
    -> FROM FLIGHT
    -> WHERE source != 'Kathmandu';
+-------------+
| noFlightKTM |
+-------------+
|           2 |
+-------------+
1 row in set (0.001 sec)
```

This query counts the total number of flights that do not depart from Kathmandu.

c. SQL syntax to update the fare of all flights by 2% to those flights either capacity is more than 250 or take off from Kathmandu.

Answer:

Current Data:

id	source	destination	aid	fare	discount	capacity
3L046	Kathmandu	Abu Dhabi	3	70000.00	(NULL)	300
AI215	Delhi	Kathmandu	2	60000.00	5000.00	300
H9891	Kuala Lumpur	Kathmandu	4	65000.00	3000.00	250
RA230	Kathmandu	Delhi	1	50000.00	(NULL)	200
RA416	Kathmandu	Kuala Lumpur	1	55000.00	2000.00	180
(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

```
UPDATE FLIGHT
SET fare = fare * 1.02
WHERE capacity > 250
  OR source = 'Kathmandu';
```

```
MariaDB [TRACKER]> UPDATE FLIGHT
    -> SET fare = fare * 1.02
    -> WHERE capacity > 250 OR source = 'Kathmandu';
Query OK, 4 rows affected (0.005 sec)
Rows matched: 4  Changed: 4  Warnings: 0
```

The query appends the fare by 2% for selected flights.

After Modification:

id	source	destination	aid	fare	discount	capacity
3L046	Kathmandu	Abu Dhabi	3	71400.00	(NULL)	300
AI215	Delhi	Kathmandu	2	61200.00	5000.00	300
H9891	Kuala Lumpur	Kathmandu	4	65000.00	3000.00	250
RA230	Kathmandu	Delhi	1	51000.00	(NULL)	200
RA416	Kathmandu	Kuala Lumpur	1	56100.00	2000.00	180
* (NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

Multiple Times:

id	source	destination	aid	fare	discount	capacity
3L046	Kathmandu	Abu Dhabi	3	74284.56	(NULL)	300
AI215	Delhi	Kathmandu	2	63672.48	5000.00	300
H9891	Kuala Lumpur	Kathmandu	4	65000.00	3000.00	250
RA230	Kathmandu	Delhi	1	53060.40	(NULL)	200
RA416	Kathmandu	Kuala Lumpur	1	58366.44	2000.00	180
* (NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

When the query was implemented multiple times, different O/P was observed because each execution increased the fare again by 2% on the already updated value.

Therefore, the query should be implemented only once to maintain appropriate results.

d. SQL query to find the fare and capacity of all flights whose discount is not available with destination having at least 5 characters.

Answer:

SELECT fare, capacity

FROM FLIGHT

WHERE discount IS NULL AND CHAR_LENGTH (destination) >= 5;

```
MariaDB [TRACKER]> SELECT fare, capacity
    -> FROM FLIGHT
    -> WHERE discount IS NULL
    -> AND CHAR_LENGTH(destination) >= 5;
+----------+----------+
| fare     | capacity |
+----------+----------+
| 74284.56 |      300 |
| 53060.40 |      200 |
+----------+----------+
2 rows in set (0.001 sec)
```

This query gets the fare and capacity of flights where no discount is given and the destination name has at least 5 characters.

Tools and Technologies Used:

1. XAMPP Server
   I used **XAMPP Server** to run a local server environment in order to manage the database in the system. I also used its terminal to perform certain SQL task like connecting it to **Localhost** and many more.

2. SQLyog
   I used **SQLyog** to work on the database i.e. create tables, and run certain SQL queries to check and manage the data in the database.

3. VS Code
   I used **VS Code** to work on the coding part precisely to build connection between the database and the python. SQL part was also coded here for the record.

4. Jupyter Notebook (VS Code)
   I used **Jupyter Notebook** within VS Code to write and execute python scripts precisely to build the connection with the database and inserting data into the database of the system.